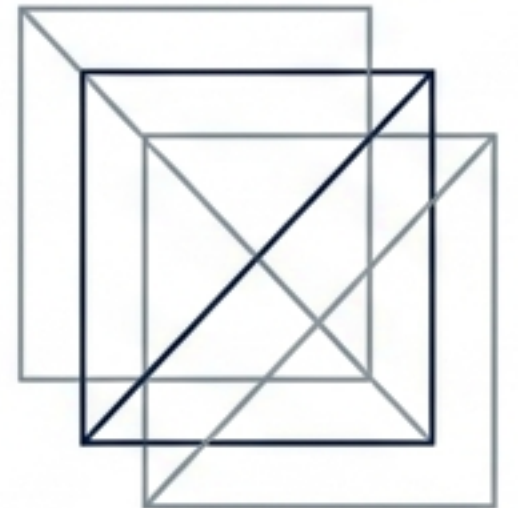


Requirements Engineering

Defining the Services and Constraints of Software Systems



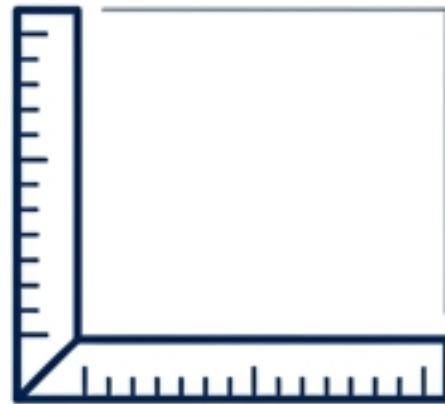
What is a Requirement?

Requirements engineering discovers, analyzes, documents, and checks the blueprint of a system.



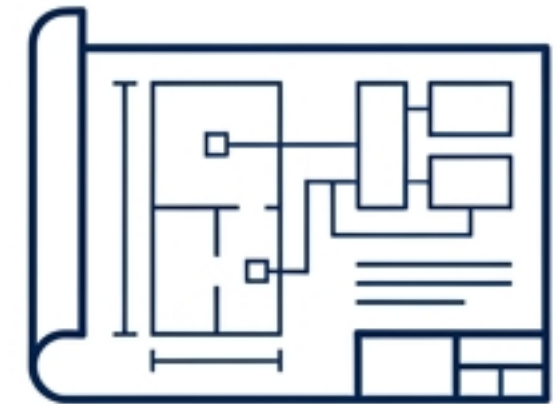
Services

(What it must do)



Constraints

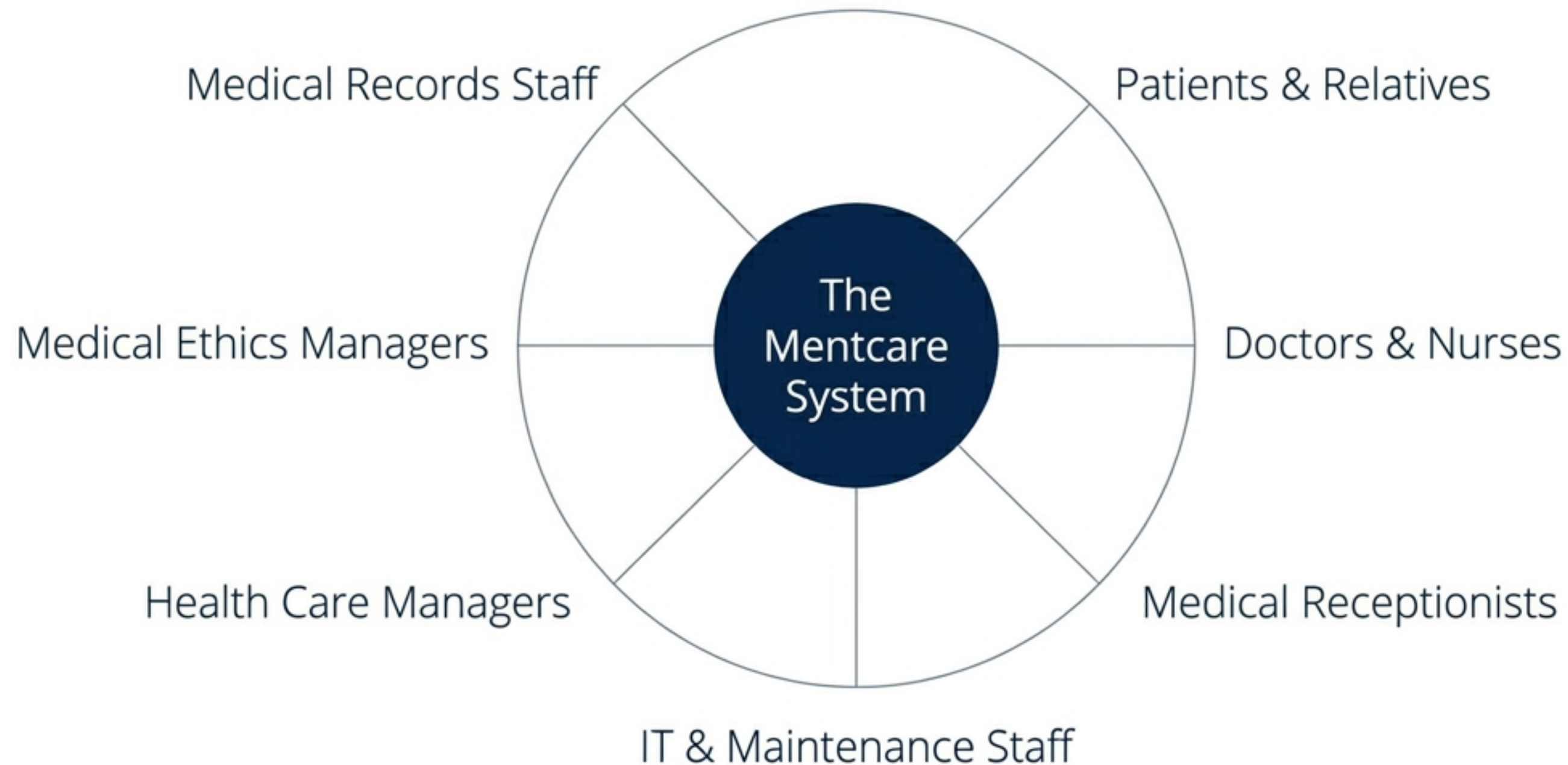
(Limits on its operation)



System
Requirements

The System Ecosystem

Requirements must address the needs of anyone with a legitimate interest in the system.



Distinct Views for Distinct Audiences

Requirements are written at different levels of detail because readers use them in different ways.

User Requirements

- Abstract, broad statements of required features.
- Written in natural language plus diagrams.

System Requirements

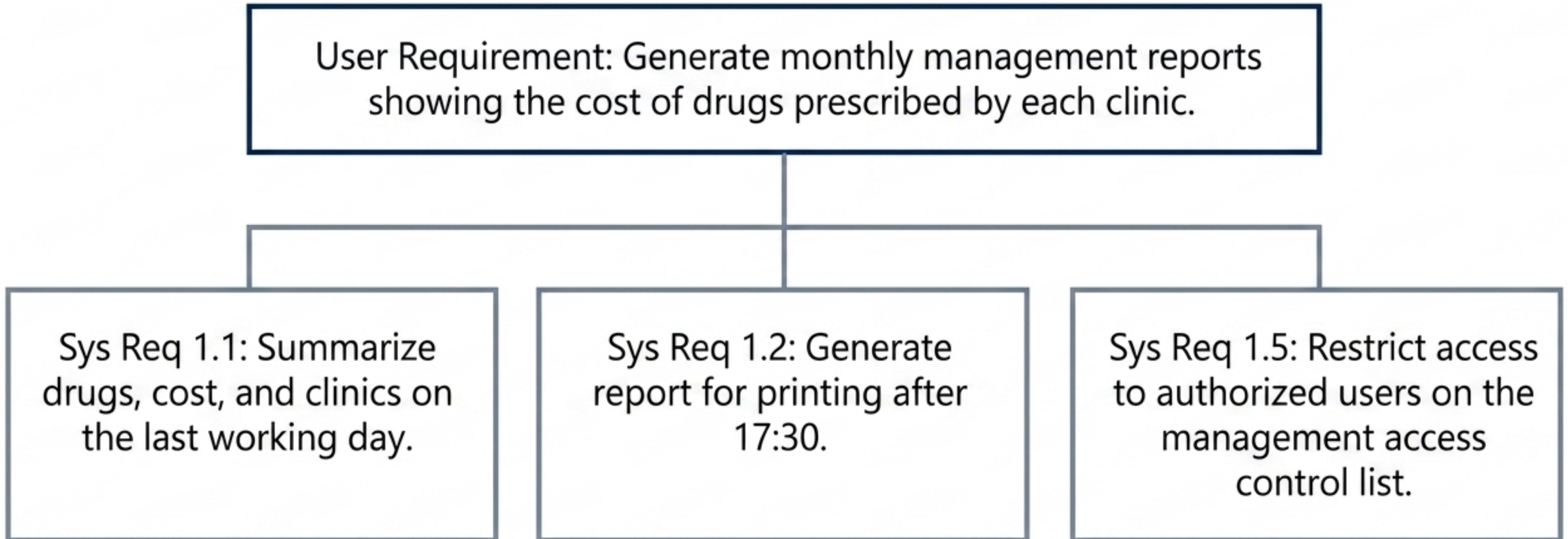
- Detailed, precise definitions of exact implementation.
- Acts as a functional specification contract for developers.

The Abstraction Matrix

	User	System
Focus	High-level needs	Exact implementation
Format	Natural language + diagrams	Functional specification contracts
Audience	Client managers, end-users	Software developers, architects
Goal	What the system provides	How it operates

Expanding the Requirement

One broad user requirement generates multiple specific system requirements.



The Two Categories of Requirements

Takeaway: Every system is defined by what it does and how it behaves.



Functional Requirements

Definition:

Services the system provides.

Focus:

Actions, inputs, outputs, exceptions.



Non-Functional Requirements

Definition:

Constraints on the system.

Focus:

Timing, performance, standards, process.

The Functional Requirement Trap

Ambiguity leads to incorrect implementations and costly delays.



Complete

(All user services defined)

Consistent

(No contradictory definitions)

Developers interpret vague requirements—like “search all clinics”—in the easiest way to implement, which often bypasses the user’s actual need.

The Hidden Dealbreakers

Users can work around a broken function, but a failed non-functional requirement makes the whole system unusable.



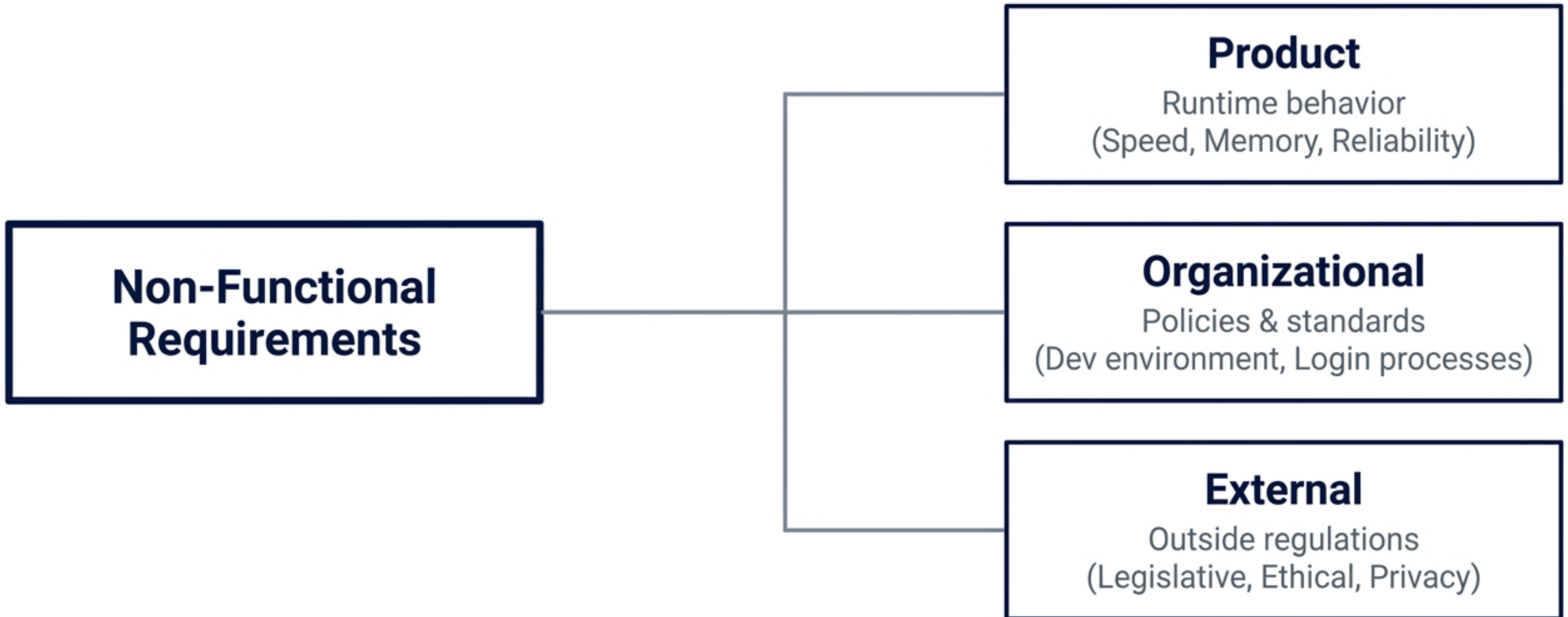
Failing to meet a non-functional requirement can mean that the whole system is unusable.



-
- Uncertified safety systems (e.g., Aircraft reliability)
 - Failed performance constraints (e.g., Embedded control timing)

Origins of System Constraints

Constraints originate from the product itself, the organization, or the external world.



The Mentcare Constraints

How the three constraint origins apply directly to medical software.

Product

Downtime within normal working hours shall not exceed 5 seconds in any one day.

Organizational

Users shall identify themselves using their health authority identity card (swipe-card).

External

The system shall implement patient privacy provisions as set out in national standard HStan-03-2006-priv.

From Goals to Metrics

Takeaway: Non-functional requirements must be written **quantitatively** to be objectively tested.



The system should be easy to use by medical staff and user errors are minimized.



After two hours of training, the average number of errors made by experienced users shall not exceed two per hour of system use.

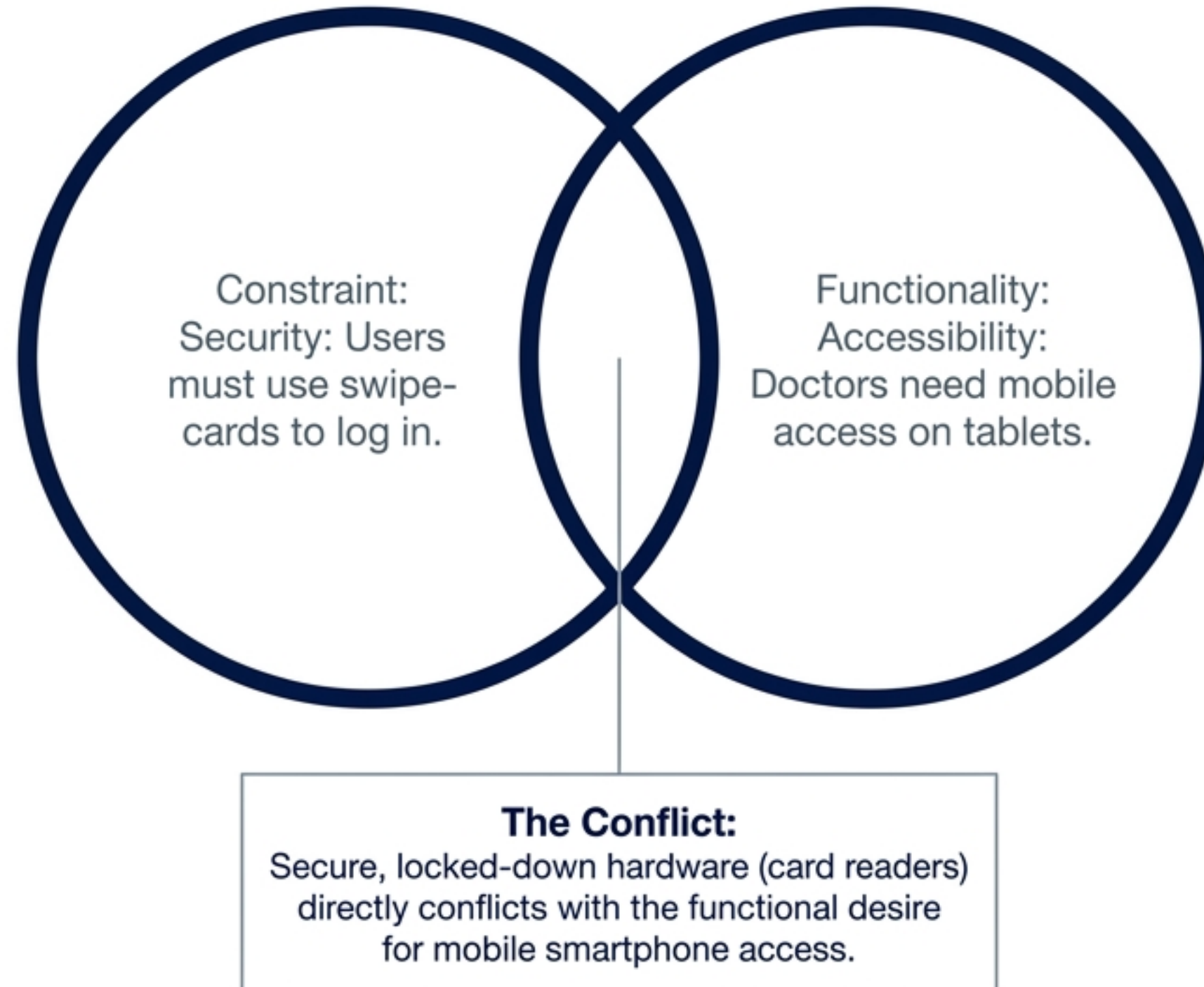
Metrics for Objective Testing

Replace subjective adjectives with measurable units.

Property	Measure
Speed	Transactions/second, User/event response time
Size	Megabytes, Number of ROM chips
Ease of Use	Training time, Number of help frames
Reliability	Mean time to failure, Availability rate

The Requirements Ecosystem

Requirements are not independent. Constraints generate new functional features and can conflict with other goals.



Summary

1.

Abstraction matters:

Separate high-level user goals from precise system specifications.

2.

Define both realms:

Document both the Functional (what the system does) and Non-Functional (how it behaves).

3.

Measure everything:

Vague goals lead to implementation disputes; objective metrics ensure testable success.